

CAPSTONE PROJECT

AI-powered Resume Screening and Ranking System

PRESENTED BY

**STUDENT NAME: CHANDRA MAULESHWAR
UPADHYAY**

**COLLEGE NAME: DEEN DAYAL UPADHYAYA
GORAKHPUR UNIVERSITY**

DEPARTMENT: COMPUTER SCIENCE

**EMAIL ID:
CHANDRAMAULESHWARUPADHYAYUPADH@GMAIL.
COM**

AICTE STUDENT ID: STU67630F029B71C1734545154



OUTLINE

- Problem Statement
- Proposed System/Solution
- System Development Approach
- Algorithm & Deployment
- Result (Output Image)
- Conclusion
- Future Scope
- References

PROBLEM STATEMENT

In the modern recruitment process, hiring managers and HR professionals are often overwhelmed by the sheer volume of resumes they receive for a single job opening. Manually reviewing each resume to identify the most relevant candidates is not only time-consuming but also increases the likelihood of errors, inconsistencies, and unconscious bias. This leads to inefficient hiring decisions and delays in the recruitment cycle.

Traditional methods of resume screening lack the ability to accurately and consistently assess how well a candidate's qualifications align with a specific job description. As a result, potentially strong candidates may be overlooked, while less suitable ones may be shortlisted.

To address this challenge, there is a pressing need for an automated and intelligent system that can assist recruiters by analyzing the textual content of resumes, comparing them with the job requirements, and ranking them based on relevance. Such a system should streamline the initial screening process, reduce human effort, improve objectivity, and enhance the overall efficiency of talent acquisition.

PROPOSED SOLUTION

To address the inefficiencies in manual resume screening, this project proposes the development of an AI-powered Resume Screening and Candidate Ranking System using Python and Streamlit. The system automates the process of evaluating resumes by leveraging Natural Language Processing (NLP) techniques to compare candidate resumes with a given job description.

The solution allows recruiters to input a job description and upload multiple resumes in PDF format. It then extracts the textual content from each resume using the PyPDF2 library. The job description and all resume texts are converted into numerical representations using the TF-IDF (Term Frequency–Inverse Document Frequency) vectorization technique, which captures the importance of words relative to the document set.

Using cosine similarity, the system calculates how closely each resume matches the job description. Based on the similarity scores, resumes are ranked in descending order of relevance. The final output is a tabular display showing each resume file name alongside its relevance score, helping recruiters to quickly identify the most suitable candidates.

The solution is implemented in a user-friendly Streamlit web interface, making it accessible even to non-technical users. It significantly reduces the time and effort involved in resume screening, while improving consistency, fairness, and decision-making accuracy in the recruitment process.

SYSTEM APPROACH

1. Input:

Recruiter enters a job description and uploads multiple PDF resumes via the Streamlit interface.

2. Text Extraction:

Text is extracted from each resume using the PyPDF2 library.

3. Text Vectorization:

Job description and resumes are converted into numerical vectors using TF-IDF (Term Frequency–Inverse Document Frequency).

4. Similarity Calculation:

Cosine Similarity is used to measure how closely each resume matches the job description.

5. Ranking & Output:

Resumes are ranked based on similarity scores and displayed in a sorted table, helping recruiters shortlist candidates efficiently.

ALGORITHM & DEPLOYMENT

Algorithm

Technique: TF-IDF + Cosine Similarity (Unsupervised Learning)

Steps:

1. Extract text from resumes and job description
2. Convert text to vectors using TF-IDF
3. Compute similarity using Cosine Similarity
4. Rank resumes based on similarity scores

Deployment

Platform: Streamlit Web App

Tools: Python, PyPDF2, scikit-learn, pandas

Run Locally: `streamlit run app.py`

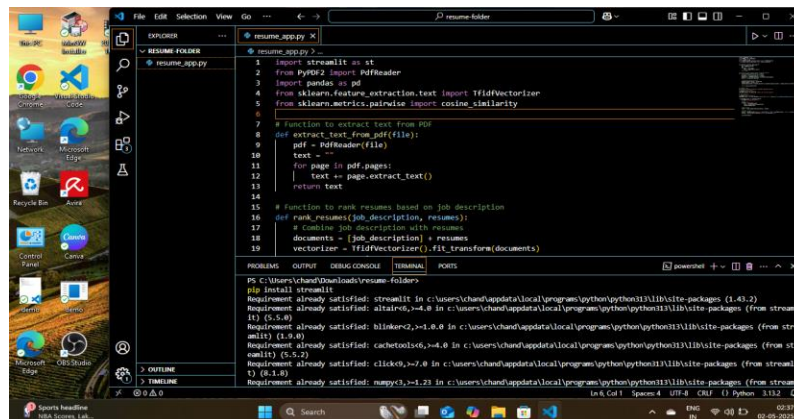
Deploy Online: Streamlit Cloud / Heroku / AWS

RESULT

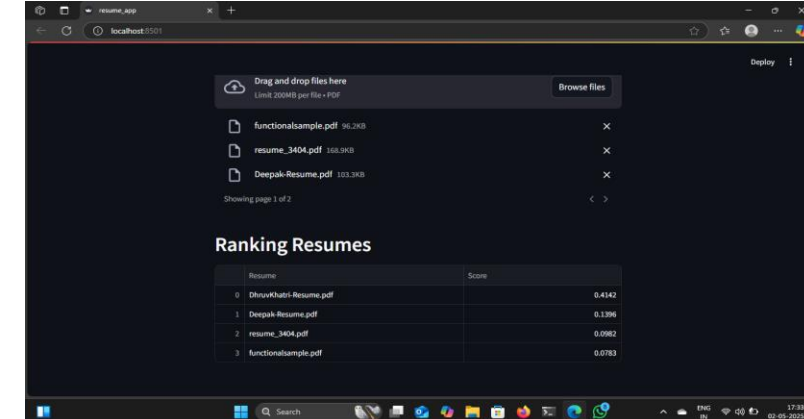
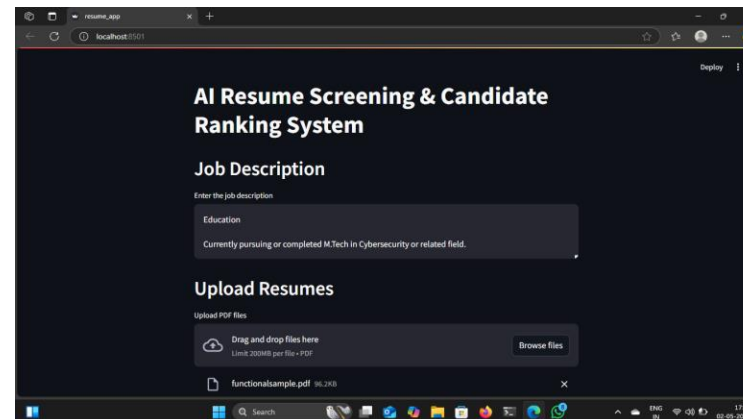
FIG.1: It represents the source code that is used in this project it contains various types of libraries including Machine Learning.

FIG.2: It represents the very first interface which we get right after executing the source code where we give qualification along with some resume and it will show us the respective result based on that input.

FIG.3: It represents the ranking output of the resumes based on the best match to the qualifications in range 0 to 1 in a sequence .



```
1 import streamlit as st
2 from PyPDF2 import PdfReader
3 import pandas as pd
4 from sklearn.feature_extraction.text import TfidfVectorizer
5 from sklearn.metrics.pairwise import cosine_similarity
6
7 # Function to extract text from PDF
8 def extract_text_from_pdf(file):
9     pdf = PdfReader(file)
10     text = ""
11     for page in pdf.pages:
12         text += page.extract_text()
13     return text
14
15 # Function to rank resumes based on job description
16 def rank_resumes(job_description, resumes):
17     # Combine job description with resumes
18     documents = [job_description] + resumes
19     vectorizer = TfidfVectorizer().fit_transform(documents)
```



CONCLUSION

The implemented AI-based Resume Screening and Candidate Ranking System provides an efficient solution to the manual and time-consuming task of shortlisting candidates. By leveraging TF-IDF vectorization and cosine similarity, the system effectively analyzes and compares resume content against a job description to rank candidates based on relevance.

Built with Python and Streamlit, the application offers a simple and interactive interface that allows recruiters to upload resumes and instantly receive ranked results. The system is lightweight, unsupervised, and requires no prior training data, making it practical and easy to deploy.

Overall, this project demonstrates the potential of natural language processing techniques in streamlining HR workflows and improving decision-making in the hiring process.

FUTURE SCOPE

OCR Integration:

Add support for scanned/image-based resumes using Optical Character Recognition (OCR) tools like Tesseract.

Advanced NLP Techniques:

Implement deep learning models like BERT for better contextual understanding and more accurate matching.

Skill & Keyword Extraction:

Automatically extract and highlight key skills, certifications, and keywords from resumes.

Support for Multiple File Formats:

Enable parsing of DOCX, TXT, and other resume formats in addition to PDF.

Candidate Insights:

Generate a summary or ranking explanation to help recruiters understand why a resume was ranked higher.

Admin Dashboard & Analytics:

Provide a dashboard to view ranking history, application trends, and candidate statistics.

REFERENCES

Python Official Documentation – <https://docs.python.org/>

Streamlit Documentation – <https://docs.streamlit.io/>

PyPDF2 Library – <https://pypi.org/project/PyPDF2/>

Scikit-learn Documentation – <https://scikit-learn.org/>

GitHub Link:

https://github.com/satyam2003-cpu/AI-powered_resume_screening.git

Thank you

